RTK 项目工作日志

GNSS 实时相对定位 (RTK)

RprPayador

2026 年 9 月

目录

1 2026-09-15 星期二 2

1.1 feat#1 INI 配置与时间同步 2
1.2 issue#1 基站解码过冲导致双站历元丢失 2
1.3 fix of issue#1 基站解码逻辑改为追平即返回 2
1.4 issue#2 文件末尾最后 4 个历元丢失 4
1.5 fix of issue#2 文件读取完毕后增加缓冲区读完逻辑 5

2 2026-09-16 星期三 7

2.1 feat#1 LockTime 失锁检测 7
2.2 feat#2 站间单差观测值构建与周跳检验 7

3 2026-09-22 星期二 14

3.1 feat#1 双差基准星选取与双差卫星数统计 14
3.2 feat#2 相对定位双差浮点解算法推导与实现 14
3.2.1 一、原始观测方程与双差消元推导 14
3.2.2 二、几何观测方程的 Taylor 展开与线性化 15
3.2.3 三、函数模型矩阵形式：$\mathbf{Z} = \mathbf{HX} + \boldsymbol{\Delta}$ 16
3.2.4 四、随机模型构建与闭合解析权矩阵 18
3.2.5 五、最小二乘估计求解与精度评定流程 19
3.2.6 六、RTKFloat 核心算法 21
3.3 issue#1 流动站高程异常漂移导致 4097 历元对流层模型崩溃 27
3.4 fix of issue#1 增加卫星健康度检查与对流层模型边界保护 28

4 2026-09-23 星期三 30

4.1 feat#3 整周模糊度固定与 LAMBDA 算法原理 30
4.1.1 一、整数最小二乘准则 (ILS) 30
4.1.2 二、LAMBDA 降相关与整数 Z 变换 30
4.1.3 三、条件方差分解与收缩椭球搜索 (MLAMBDA) 31
4.2 feat#4 RTKLIB 开源算法的解耦与工程适配改造 31
4.2.1 一、开源来源与接口对齐 31
4.2.2 二、消除外部 BLAS/LAPACK 与动态库依赖 32
4.2.3 三、矩阵运算与内存管理自治化 33
4.3 feat#5 模糊度 Ratio 检验与固定基线解算公式推导 33
4.3.1 一、Ratio 检验统计量 33
4.3.2 二、固定基线向量更新与方差恢复推导 34
4.4 arch 相对定位主流程中的调度与接口规划 34

1 2026-09-15 星期二

1.1 feat#1 INI 配置与时间同步

(1) 新增 ReadConfig.cpp 与 config.ini, 程序改为通过 INI 配置文件统一管理运行模式、数据路径与质量控制参数, 同时添加了两种缺省设置，在不传入配置文件路径作为参数的情况下在根目录下寻找config.ini作为配置文件，如果文件不存在则使用默认参数，并在 ROVERCFGINFO 中加入 IsRTKMode 成员实现 SPP/RTK 模式分流;

(2) 编写 Sync.cpp 的 GetSynObs() 函数, 实现基准站与流动站观测历元的时间同步 (RTK 数据流第一步);

(3) 重构 RTK.cpp 主函数: 拆分为 RunSPP() 与 RunRTK() 两个分支函数,SPP 分支逻辑保持不变;

1.2 issue#1 基站解码过冲导致双站历元丢失

使用两份相同的数据文件作为流动站和基站的数据（./data/oem719-202603111200.bin，共 1077 历元），进行同步测试：

预期结果：1077 个历元全部同步成功

实际结果：536 个历元同步成功, 且同步成功的历元间时间间隔均为 2

查明原因：

问题出在 GetSynObs() 读基站的内层循环：**基站时刻追平流动站后，解码循环不会停止**。

原逻辑中，基站每解码出一个历元就重算一次时差 dt（流动站时刻减基站时刻），但循环内只在 dt < -SynThres 时才返回，dt 已追平（dt ≤ SynThres）时既不返回也不退出循环，而是继续解码缓冲区中剩余的消息。由于 fread 每次读取 40KB（约含 5～7 个完整历元），基站解码必然**越过流动站当前时刻发生“过冲”**，使得 dt 变为负值且超过阈值，触发 return -1。

而 return -1 的代价是**丢弃刚解码出的流动站历元**（Raw->RovEpk 将在下次调用中被覆盖）。丢弃后，基站已停在过冲位置，下次调用时流动站解出的下一个历元恰好与已过冲的基站时刻对齐，直接配对成功。于是每两个历元就有一个在过冲时被作废：同步历元间隔恒为 2 秒，同步总数 536 ≈ 1077/2。首个同步历元为文件第 2 个历元（TOW 273996.000），第 1 个历元在首次追平时被同样机制丢弃。

1.3 fix of issue#1 基站解码逻辑改为追平即返回

将基站解码循环改为**追平即返回**:解码出一个基站历元后若 abs(dt) ≤ SynThres，立即 return 1，不再继续解码缓冲区。修复后实测同步 1073/1077 个历元，相邻同步历元间隔全部恢复为 1 秒，首历元（TOW 273995.000）正常同步。

关键代码：

```
int GetSynObs(FILE* FBas, FILE* FRov, ...)
{
    // 比较流动站时刻与上次基站时刻
    double dt = GetDiffTime(&Raw->RovEpk.Time, &Raw->BasEpk.Time);

    // 流动站比基站早，说明基站还没跟上，暂时无法同步
    if (dt < -SynThres) return -1;
    if (abs(dt)<=SynThres) return 1;

    // 如果基站时刻已经同步（或基站未初始化 dt 很大），进入读基站
    // 读基站，直到基站时刻追上流动站
    while (dt > SynThres) {
        if (IsFileData == 1) {
            BasReadLen = fread(BasBuff + BasLen, 1, MAXRAWLEN - BasLen, FBas);
            if (BasReadLen == 0) return 0;
        }
        else {
            BasReadLen = recv(BasSock, (char*)(BasBuff + BasLen), MAXRAWLEN - BasLen, 0);
            if (BasReadLen == 0) return 0;
        }
        BasLen += BasReadLen;

        while (true) {
            val = DecodeNovOem7Dat(BasBuff, BasLen, &Raw->BasEpk, Raw->GpsEph, Raw->BdsEph);
            if (val == -1) break;
            if (val == 1) {
                dt = GetDiffTime(&Raw->RovEpk.Time, &Raw->BasEpk.Time);
                if (dt < -SynThres) return -1;
                else if(abs(dt)<=SynThres) return 1;
            }
        }
    }
```

```
}
```

1.4 issue#2 文件末尾最后 4 个历元丢失

在修复过冲错误后，重新运行程序，运行结果：

```
Config file 'config.ini' loaded successfully.
Starting RTK Debugging Flow...
Sync epoch 1: TOW 273995.000, Bas 26 sats, Rov 26 sats
Sync epoch 2: TOW 273996.000, Bas 26 sats, Rov 26 sats
Sync epoch 3: TOW 273997.000, Bas 26 sats, Rov 26 sats
Sync epoch 4: TOW 273998.000, Bas 26 sats, Rov 26 sats
Sync epoch 5: TOW 273999.000, Bas 26 sats, Rov 26 sats
Sync epoch 6: TOW 274000.000, Bas 26 sats, Rov 26 sats
Sync epoch 7: TOW 274001.000, Bas 26 sats, Rov 26 sats
Sync epoch 8: TOW 274002.000, Bas 26 sats, Rov 26 sats
Sync epoch 9: TOW 274003.000, Bas 26 sats, Rov 26 sats
Sync epoch 10: TOW 274004.000, Bas 26 sats, Rov 26 sats
Sync epoch 11: TOW 274005.000, Bas 26 sats, Rov 26 sats
...
Sync epoch 1070: TOW 275064.000, Bas 28 sats, Rov 28 sats
Sync epoch 1071: TOW 275065.000, Bas 28 sats, Rov 28 sats
Sync epoch 1072: TOW 275066.000, Bas 28 sats, Rov 28 sats
Sync epoch 1073: TOW 275067.000, Bas 28 sats, Rov 28 sats

RTK Processing Summary:
Total Synced Epochs: 1073
Results saved to ./result.txt
```

issue#1 已修复，但是全部 1077 个历元，最后 4 个历元没有输出结果

定位问题：关键代码：

```
    while (!RovDone) {
        if (IsFileData == 1) {
            RovReadLen = fread(RovBuff + RovLen, 1, MAXRAWLEN -
                RovLen, FRov);
            if (RovReadLen == 0) return 0;
        } else {
            Sleep(980);
            RovReadLen = recv(RovSock, (char*)(RovBuff + RovLen)
                , MAXRAWLEN - RovLen, 0);
            if (RovReadLen == 0) return 0;
        }
        RovLen += RovReadLen;

        while (true) {
            val = DecodeNovOem7Dat(RovBuff, RovLen, &Raw->RovEpk
                , Raw->GpsEph, Raw->BdsEph);
```

```
            if (val == -1) break;           // 帧不完整，回去继续读
            if (val == 1) { RovDone = true; break; }  // 解出观测历元
        }
    }
```

在这段代码的逻辑中，当从文件中读取的数据长度为 0 时，就返回 0 了（已读到文件末尾），但实际上，这只说明文件里的所有内容已经被读取到RovBuff，而缓冲区中的数据可能并没有被全部解码

基站数据同理

1.5 fix of issue#2 文件读取完毕后增加缓冲区读完逻辑

修复思路：fread/recv 返回 0（读到文件末尾）时，不能立即 return 0，而应当先**冲刷缓冲区**，把缓冲区内剩余的完整消息继续解码，直到解出历元或缓冲区彻底解空；基站侧同理，冲刷阶段解出历元后照常计算 dt 进行配对。

关键代码：

```
while (!RovDone) {
    if (IsFileData == 1) {
        RovReadLen = fread(RovBuff + RovLen, 1, MAXRAWLEN -
            RovLen, FRov);
        if (RovReadLen == 0) {//文件读完
            while (true) {   // 冲刷：把缓冲区剩余的完整历元解出来
                val = DecodeNovOem7Dat(RovBuff, RovLen, &Raw->
                    RovEpk, Raw->GpsEph, Raw->BdsEph);
                if (val == -1) return 0;          // 缓冲区彻底解空
                if (val == 1) { RovDone = true; break; }
            }
            break;
        }
    } else { ... }
    ...
}
```

调试记录：第一版冲刷代码中 break 未放入 if 的大括号内，导致冲刷阶段解出星历消息时也无条件跳出循环，拿着上一历元的旧数据去配对，出现同一历元被同步两次的现象（TOW 275071 连续出现两次）；补上大括号后消失。

验证结果：1077/1077 个历元全部同步成功，相邻历元间隔全部为 1 秒，首历

元 TOW 273995.000、末历元 TOW 275071.000，无重复、无缺失。

```
Sync epoch 1: TOW 273995.000, Bas 26 sats, Rov 26 sats
...
Sync epoch 1077: TOW 275071.000, Bas 28 sats, Rov 28 sats

RTK Processing Summary:
Total Synced Epochs: 1077
```

2 2026-09-16 星期三

2.1 feat#1 LockTime 失锁检测

Parity 的判断逻辑已在解码函数中实现：当奇偶标志（parity_known）无效时，将载波相位观测值置 0.0，交由后级的四零检查淘汰。

原计划在 DetectOutlier() 中添加根据 LockTime 判断卫星观测值是否可用的逻辑，再在 FormSDEpochObs() 中直接调用它实现周跳探测。**但发现一个问题**：DetectOutlier() 的参数中只有本历元的 EPOCHOBS，其中 ComObs 仅保存了上一历元的 MW/GF/PIF/n，而携带 LockTime 与 half 的 SATOBS 每个历元都会被解码器重置——函数内部拿不到上一历元的质量标志，跨历元比对无从谈起。

解决方法：在 FormSDEpochObs.cpp 内部维护静态历史数组保存上一历元各卫星基站和流动站的 LockTime。这样既不需要改动公共头文件 RTK_Structs.h 里的结构体定义，又能原生使用 double 精度直接比对，对中途失锁或隔历元重现的卫星做到了天然防御。

2.2 feat#2 站间单差观测值构建与周跳检验

在 FormSDEpochObs.cpp 中实现站间单差观测值构建函数 FormSDEpochObs() 与单差周跳检验函数 DetectCycleSlip()，完成基站与流动站同步历元观测值的单差匹配、硬件失锁判定、单差组合观测值计算以及周跳检验。

功能设计与实现逻辑：

(1) **站间单差观测值构建（FormSDEpochObs）：**清空当前历元单差卫星数组 SdSatObs（保留 SdCObs 用于周跳平滑）。遍历流动站卫星，在基准站中查找系统号和 PRN 相同的共视卫星。流动站和基站均进行双频四零检查（$c_1, l_1, p_2, l_2 \neq 0$），剔除观测值不全的卫星。对双频齐全的共视卫星求站间差（流动站 − 基站）：$dP = P_{Rov} - P_{Bas}$，$dL = L_{Rov} - L_{Bas}$，并记录其在流动站与基站中的原始索引 nRov 和 nBas。

(2) **基于内部静态数组的 LockTime 失锁检测：**在 FormSDEpochObs 内部定义静态数组 preLockTimeRov[6][64][2]、preLockTimeBas[6][64][2] 与跟踪状态标志 hasPre[6][64]。对于上一历元已跟踪的卫星，比对基站与流动站的双频锁定时间；若任一站任一频点锁定时间发生回退（小于上一历元值），判定发生硬件失锁，置标志位 sdSat.Valid = 0，否则置为 1；若为新接入卫星则置 sdSat.Valid = 1。最后更新内部静态历史数组。

(3) **单差 GF 与 MW 组合观测值计算（DetectCycleSlip）：**站间求差消除了绝大部分对流层与电离层延迟，单差组合稳定性远高于非差组合。计算单差几何无关组合 $GF = dL_1 - dL_2$；根据卫星所属系统（GPS、BDS），分

别利用其对应频点的频率常数计算单差宽巷 MW 组合观测值。

(4) **单差周跳探测与状态平滑递推：**从上一历元单差平滑缓冲区 SdCObs 中匹配同系统同 PRN 的历史状态。若前级 LockTime 判定失锁（sdSat.Valid == 0），或者历元间单差几何无关组合跳变 $|dGF| > 0.05\,\mathrm{m}$、MW 组合跳变 $|dMW| > 3.0$ 周，判定为周跳或粗差：标记 sdSat.Valid = 0，并以当前历元 MW 值重置平滑弧段（平滑计数 $n = 1$）；若连续无周跳，标记 sdSat.Valid = 1，按递推加权公式平滑 MW 组合并累加计数 n；新接入卫星初始化 $n = 1$。处理完成后将当前历元平滑状态写入 Obs->SdCObs 持久化保存。

核心代码实现：

```
void FormSDEpochObs(const EPOCHOBS *EpkBas, const EPOCHOBS *
    EpkRov, SDEPOCHOBS *SDObs){
    if (!EpkBas || !EpkRov || !SDObs) return;

    static double preLockTimeRov[6][64][2] = {0};
    static double preLockTimeBas[6][64][2] = {0};
    static bool   hasPre[6][64] = {false};

    // 清空单差卫星观测数据
    memset(SDObs->SdSatObs, 0, sizeof(SDObs->SdSatObs));
    int sdNum = 0;

    // 根据流动站卫星数进行循环
    for(int i = 0; i < EpkRov->SatNum; i++){
        const SATOBS& satRov = EpkRov->SatObs[i];

        // 1. 检查流动站卫星号和系统号是否正常，双频观测值是否完
            整
        if (satRov.Prn <= 0 || (satRov.System != GPS && satRov.
            System != BDS)) continue;
        if (satRov.c1 == 0.0 || satRov.l1 == 0.0 || satRov.p2 ==
             0.0 || satRov.l2 == 0.0) continue;

        // 2. 在基站观测值中查找与该流动站相同的卫星
        int idxBas = -1;
        for(int j = 0; j < EpkBas->SatNum; j++){
            if (EpkBas->SatObs[j].Prn == satRov.Prn && EpkBas->
                SatObs[j].System == satRov.System){
                idxBas = j;
```

```
                break;
            }
        }
        if (idxBas == -1) continue;

        const SATOBS& satBas = EpkBas->SatObs[idxBas];
        // 检查基站双频观测值是否完整
        if (satBas.c1 == 0.0 || satBas.l1 == 0.0 || satBas.p2 ==
            0.0 || satBas.l2 == 0.0) continue;

        // 3. 对同类型和同频率的观测值求差并保存索引号
        SDSATOBS& sdSat = SDObs->SdSatObs[sdNum];
        sdSat.Prn = satRov.Prn;
        sdSat.System = satRov.System;
        sdSat.nRov = (short)i;
        sdSat.nBas = (short)idxBas;

        // 站间单差: 流动站 - 基准站 (m)
        sdSat.dP[0] = satRov.c1 - satBas.c1;
        sdSat.dL[0] = satRov.l1 - satBas.l1;
        sdSat.dP[1] = satRov.p2 - satBas.p2;
        sdSat.dL[1] = satRov.l2 - satBas.l2;

        // 4. LockTime 检查
        int sys = (int)satRov.System;
        int prn = satRov.Prn;

        if (sys >= 0 && sys < 6 && prn >= 0 && prn < 64) {
            if (hasPre[sys][prn]) {
                bool slipRov = (satRov.LockTime[0] <
                   preLockTimeRov[sys][prn][0] ||
                                satRov.LockTime[1] <
                                   preLockTimeRov[sys][prn][1]);
                bool slipBas = (satBas.LockTime[0] <
                   preLockTimeBas[sys][prn][0] ||
                                satBas.LockTime[1] <
                                   preLockTimeBas[sys][prn][1]);

                if (slipRov || slipBas) {
                    sdSat.Valid = 0;
```

```
                } else {
                    sdSat.Valid = 1;
                }
            } else {
                sdSat.Valid = 1; // 新卫星接入
            }

            // 更新静态历史
            preLockTimeRov[sys][prn][0] = satRov.LockTime[0];
            preLockTimeRov[sys][prn][1] = satRov.LockTime[1];
            preLockTimeBas[sys][prn][0] = satBas.LockTime[0];
            preLockTimeBas[sys][prn][1] = satBas.LockTime[1];
            hasPre[sys][prn] = true;
        } else {
            sdSat.Valid = 1;
        }

        sdNum++;
        if (sdNum >= MAXCHANNUM) break;
    }

    // 循环结束后，对单差观测值的观测时刻和卫星数量赋值
    SDObs->Time = EpkRov->Time;
    SDObs->SatNum = (short)sdNum;
}

// 单差周跳探测函数
void DetectCycleSlip(SDEPOCHOBS* Obs){
    if (!Obs) return;

    // 备份上一历元单差组合平滑状态
    MWGF PreSdCObs[MAXCHANNUM];
    memcpy(PreSdCObs, Obs->SdCObs, sizeof(PreSdCObs));

    // 准备当前历元记录
    MWGF CurSdCObs[MAXCHANNUM];
    memset(CurSdCObs, 0, sizeof(CurSdCObs));

    for (int i = 0; i < Obs->SatNum; i++) {
        SDSATOBS& sdSat = Obs->SdSatObs[i];
```

```
100
101         CurSdCObs[i].Prn = sdSat.Prn;
102         CurSdCObs[i].Sys = sdSat.System;
103
104         // 1. 计算单差组合观测值
105         double GF = sdSat.dL[0] - sdSat.dL[1]; // 单差几何无关组
                合 (m)
106         double MW = 0.0;                       // 单差 MW 组合 (
                周)
107         if (sdSat.System == GPS) {
108             MW = (FG1_GPS * sdSat.dL[0] - FG2_GPS * sdSat.dL[1])
                    / (FG1_GPS - FG2_GPS)
109                - (FG1_GPS * sdSat.dP[0] + FG2_GPS * sdSat.dP[1])
                       / (FG1_GPS + FG2_GPS);
110         } else if (sdSat.System == BDS) {
111             MW = (FG1_BDS * sdSat.dL[0] - FG3_BDS * sdSat.dL[1])
                    / (FG1_BDS - FG3_BDS)
112                - (FG1_BDS * sdSat.dP[0] + FG3_BDS * sdSat.dP[1])
                       / (FG1_BDS + FG3_BDS);
113         }
114
115         // 2. 寻找上一历元单差组合记录
116         int preIdx = -1;
117         for (int j = 0; j < MAXCHANNUM; j++) {
118             if (PreSdCObs[j].Prn == sdSat.Prn && PreSdCObs[j].
                    Sys == sdSat.System) {
119                 preIdx = j;
120                 break;
121             }
122         }
123
124         // 3. 周跳检测与状态平滑递推
125         bool slip = (sdSat.Valid == 0); // 前级 LockTime 已标记
                失锁
126         if (preIdx != -1 && PreSdCObs[preIdx].n > 0) {
127             double dGF = fabs(GF - PreSdCObs[preIdx].GF);
128             double dMW = fabs(MW - PreSdCObs[preIdx].MW);
129
130             if (slip || dGF > 0.05 || dMW > 3.0) {
131                 // 检测到周跳或粗差: 标记无效并以当前历元重置平
```

```
                    滑
132                 sdSat.Valid = 0;
133                 CurSdCObs[i].MW = MW;
134                 CurSdCObs[i].n = 1;
135             } else {
136                 // 正常连续无周跳: 递推平滑 MW
137                 sdSat.Valid = 1;
138                 CurSdCObs[i].MW = (PreSdCObs[preIdx].n *
                        PreSdCObs[preIdx].MW + MW) / (PreSdCObs[
                        preIdx].n + 1);
139                 CurSdCObs[i].n = PreSdCObs[preIdx].n + 1;
140             }
141         } else {
142             // 新接入共视卫星, 初始化新弧段
143             sdSat.Valid = 1;
144             CurSdCObs[i].MW = MW;
145             CurSdCObs[i].n = 1;
146         }
147
148         CurSdCObs[i].GF = GF;
149     }
150
151     // 更新回持久化缓冲区
152     memcpy(Obs->SdCObs, CurSdCObs, sizeof(CurSdCObs));
153 }
```

测试验证：

在 RTK.cpp 主循环中调用 FormSDEpochObs 与 DetectCycleSlip，利用短基线实测数据进行全流程测试，并将结果输出保存至 result.txt：

```
Sync epoch 1: TOW 112611.000, Bas 24 sats, Rov 31 sats, SD GPS 9 (Valid 9), BDS 12 (Valid 12)
Sync epoch 2: TOW 112612.000, Bas 25 sats, Rov 31 sats, SD GPS 9 (Valid 9), BDS 12 (Valid 12)
Sync epoch 3: TOW 112613.000, Bas 25 sats, Rov 31 sats, SD GPS 9 (Valid 9), BDS 13 (Valid 13)
...
Sync epoch 13: TOW 112623.000, Bas 28 sats, Rov 31 sats, SD GPS 9 (Valid 9), BDS 16 (Valid 16)
...
Sync epoch 10074: TOW 122684.000, Bas 28 sats, Rov 31 sats, SD GPS 8 (Valid 8), BDS 16 (Valid 16)
Sync epoch 10075: TOW 122685.000, Bas 28 sats, Rov 31 sats, SD GPS 8 (Valid 8), BDS 16 (Valid 16)

RTK Processing Summary:
Total Synced Epochs: 10075
Results saved to ./result.txt
```

全数据集共 10075 个同步历元全部正常生成站间单差观测值。双频有效共视

卫星数稳定在 21～25 颗（GPS 8～9 颗，BDS 12～16 颗），各星单差 MW 与 GF 组合在整个时段内保持稳定递推平滑，无误判周跳，单差观测值构建与质量检测模块运行稳定。

3 2026-09-22 星期二

3.1 feat#1 双差基准星选取与双差卫星数统计

在 FormSDEpochObs.cpp 中实现双差基准星选取与双差卫星数统计函数 DetRefSat()，为构建站星双差方程确定参考基准：

(1) **基准星选取准则**：遍历有效单差星，筛选无周跳与未失锁（Valid == 1）、卫星轨道与星历计算有效（SatPVT.Valid == 1）且基站与流动站双频载噪比均充足（cn0 $>$ 35.0 dB-Hz）的候选星，在各系统（GPS、BDS）内分别选取高度角最高（Elevation 最大）的卫星作为参考星，记录其 PRN 至 DDObs->RefPrn[sys],并在单差数组中的下标记录至 DDObs->RefPos[sys]。

(2) **双差卫星数统计**：以选定的参考星为基准，统计同系统其余有效单差星数量,分别记录至 DDObs->DDSatNum[0]（GPS 双差星数 m_{DD}）与 DDObs->DDSatNum[1]（BDS 双差星数 n_{DD}），双差总星数 $S = m_{DD} + n_{DD}$。

(3) **浮点解可解性判定**：检查双差星数是否满足三维定位条件。若双差总星数 $S < 3$ 或方程总数 $l = 4S$ 小于等于待估未知数 $n = 3 + 2S$（即自由度 $r = l - n \leq 0$），则无法定出空间三维基线向量，RTKFloat 提前容错退出。

3.2 feat#2 相对定位双差浮点解算法推导与实现

在 FormSDEpochObs.cpp 中推导并实现相对定位双差浮点解算法 RTKFloat()。对站星双差观测方程、空间几何 Taylor 展开、函数模型 $\mathbf{Z} = \mathbf{HX} + \boldsymbol{\Delta}$（增量形式 $\mathbf{z} = \mathbf{Hx} + \boldsymbol{\Delta}$）、随机模型与闭合解析权矩阵 $\mathbf{W}$ 及高斯-牛顿最小二乘迭代流程进行推导与代码实现：

3.2.1 一、原始观测方程与双差消元推导

设基准站为 B（已知基准坐标 X_B, Y_B, Z_B），流动站为 R（待求坐标 X_R, Y_R, Z_R）。某一卫星系统（GPS 或 BDS）中选定高度角最高的一颗共视星为参考星（用上标 i 表示），其余有效共视星为待估双差星（用上标 j 表示）。

对于频率 f，测站 $r \in \{B, R\}$ 对卫星 $s \in \{i, j\}$ 的原始非差伪距与载波相位观测方程（长度单位：m）为：

$$P_{r,f}^s = \rho_r^s + c(dt_r - dt^s) + I_{r,f}^s + T_r^s + e_{P,r,f}^s \tag{1}$$

$$L_{r,f}^s = \rho_r^s + c(dt_r - dt^s) + \lambda_f N_{r,f}^s - I_{r,f}^s + T_r^s + \varepsilon_{L,r,f}^s \tag{2}$$

式中，$\rho_r^s = \|\mathbf{X}^s - \mathbf{X}_r\|$ 为测站到卫星的空间几何距离；dt_r 与 dt^s 分别为接收机钟差与卫星钟差；$I_{r,f}^s$ 与 T_r^s 分别为电离层延迟与对流层延迟；λ_f 为载波波长；$N_{r,f}^s$ 为整周模糊度；e, ε 为伪距与载波的随机误差。

1. 站间单差（流动站 R – 基准站 B）： 流动站与基站对同一颗卫星 s 的观测值求差，彻底消除卫星钟差 dt^s 及其初始相位偏差：

$$\Delta P_f^s = P_{R,f}^s - P_{B,f}^s = \rho_R^s - \rho_B^s + c(dt_R - dt_B) + \Delta I_f^s + \Delta T^s + \Delta e_f^s \tag{3}$$

$$\Delta L_f^s = L_{R,f}^s - L_{B,f}^s = \rho_R^s - \rho_B^s + c(dt_R - dt_B) + \lambda_f \Delta N_f^s - \Delta I_f^s + \Delta T^s + \Delta \varepsilon_f^s \tag{4}$$

其中 $\Delta N_f^s = N_{R,f}^s - N_{B,f}^s$ 为单差整周模糊度。代码中分别对应 `sdSat.dP[f]` 与 `sdSat.dL[f]`。

2. 星间双差（待估星 j – 参考星 i）： 将非参考星 j 的单差减去参考星 i 的单差，彻底消除两站间的相对接收机钟差项 $c(dt_R - dt_B)$：

$$\begin{aligned}\nabla\Delta P_f^{ij} &= \Delta P_f^j - \Delta P_f^i \\ &= (\rho_R^j - \rho_R^i) - (\rho_B^j - \rho_B^i) + \nabla\Delta I_f^{ij} + \nabla\Delta T^{ij} + \nabla\Delta e_f^{ij}\end{aligned} \tag{5}$$

$$\begin{aligned}\nabla\Delta L_f^{ij} &= \Delta L_f^j - \Delta L_f^i \\ &= (\rho_R^j - \rho_R^i) - (\rho_B^j - \rho_B^i) + \lambda_f \nabla\Delta N_f^{ij} - \nabla\Delta I_f^{ij} + \nabla\Delta T^{ij} + \nabla\Delta \varepsilon_f^{ij}\end{aligned} \tag{6}$$

3. 短基线条件简化： 在短基线定位场景（基线长度 $< 10 \sim 15\,\mathrm{km}$）下，大气延迟空间强相关，求双差后残余电离层与对流层延迟近似完全抵消：

$$\nabla\Delta I_f^{ij} \approx 0, \quad \nabla\Delta T^{ij} \approx 0$$

得到短基线经典双差观测方程：

$$\nabla\Delta P_f^{ij} = (\rho_R^j - \rho_R^i) - (\rho_B^j - \rho_B^i) + \nabla\Delta e_f^{ij} \tag{7}$$

$$\nabla\Delta L_f^{ij} = (\rho_R^j - \rho_R^i) - (\rho_B^j - \rho_B^i) + \lambda_f \nabla\Delta N_f^{ij} + \nabla\Delta \varepsilon_f^{ij} \tag{8}$$

其中 $\nabla\Delta N_f^{ij} = \Delta N_f^j - \Delta N_f^i$ 为待估的双差整周模糊度参数。

3.2.2 二、几何观测方程的 Taylor 展开与线性化

几何距离项 ρ_R^j 和 ρ_R^i 与流动站三维未知坐标 (X_R, Y_R, Z_R) 呈非线性关系，必须在近似值处进行一阶 Taylor 展开以构造线性函数模型。

1. 坐标近似值与几何距离： 基准站已知真实坐标固定为 $\mathbf{X}_B = [X_B, Y_B, Z_B]^T$，基站到各卫星的几何距离为确定常量：

$$d_B^s = \sqrt{(X^s - X_B)^2 + (Y^s - Y_B)^2 + (Z^s - Z_B)^2} \quad (s = i, j)$$

流动站真实坐标由近似值 $\mathbf{X}_R^*$ 与改正数向量 $\mathbf{x}_R = [x, y, z]^T$ 构成：

$$\mathbf{X}_R = \begin{bmatrix} X_R \\ Y_R \\ Z_R \end{bmatrix} = \begin{bmatrix} X_R^* \\ Y_R^* \\ Z_R^* \end{bmatrix} + \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \mathbf{X}_R^* + \mathbf{x}_R$$

近似值 $\mathbf{X}_R^*$ 取自当前历元流动站 SPP 解算结果。

2. 视线方向余弦向量： 流动站近似坐标到卫星 s（$s=i,j$）的坐标差与近似几何距离为：

$$\Delta X^{s*}=X^s-X_R^*,\quad \Delta Y^{s*}=Y^s-Y_R^*,\quad \Delta Z^{s*}=Z^s-Z_R^*$$

$$d_R^{s*}=\sqrt{(\Delta X^{s*})^2+(\Delta Y^{s*})^2+(\Delta Z^{s*})^2}$$

定义流动站指向卫星 s 的单位视线方向余弦为：

$$l^s=\frac{X^s-X_R^*}{d_R^{s*}},\quad m^s=\frac{Y^s-Y_R^*}{d_R^{s*}},\quad n^s=\frac{Z^s-Z_R^*}{d_R^{s*}}$$

对几何距离函数 $\rho_R^s(\mathbf{X}_R)$ 在 $\mathbf{X}_R^*$ 处展开：

$$\rho_R^s(\mathbf{X}_R)\approx d_R^{s*}-l^sx-m^sy-n^sz$$

3. 双差几何距离展开： 非参考星 j 与参考星 i 的流动站几何距离差展开为：

$$\rho_R^j-\rho_R^i\approx(d_R^{j*}-d_R^{i*})-(l^j-l^i)x-(m^j-m^i)y-(n^j-n^i)z$$

记双差视线方向余弦差（待估星 − 参考星）为：

$$\Delta l^{ij}=l^j-l^i,\quad \Delta m^{ij}=m^j-m^i,\quad \Delta n^{ij}=n^j-n^i$$

流动站与基站的双差几何距离近似计算量定义为：

$$\rho_{DD}^{ij*}=(d_R^{j*}-d_R^{i*})-(d_B^j-d_B^i)$$

则双差几何距离项线性化为：

$$(\rho_R^j-\rho_R^i)-(d_B^j-d_B^i)\approx\rho_{DD}^{ij*}-\Delta l^{ij}x-\Delta m^{ij}y-\Delta n^{ij}z$$

4. 线性化后的双差观测方程： 将上式代入短基线双差观测方程 (7) 与 (8)，并将近似计算量 ρ_{DD}^{ij*} 移至等式左端，形成观测值减计算值 OMC (Observed Minus Computed)：

$$\nabla\Delta P_f^{ij}-\rho_{DD}^{ij*}=-\Delta l^{ij}x-\Delta m^{ij}y-\Delta n^{ij}z+\nabla\Delta e_f^{ij} \tag{9}$$

$$\nabla\Delta L_f^{ij}-\rho_{DD}^{ij*}=-\Delta l^{ij}x-\Delta m^{ij}y-\Delta n^{ij}z+\lambda_f\nabla\Delta N_f^{ij}+\nabla\Delta\varepsilon_f^{ij} \tag{10}$$

3.2.3 三、函数模型矩阵形式：$\mathbf{Z}=\mathbf{HX}+\boldsymbol{\Delta}$

按照标准最小二乘估计理论，非线性观测方程经一阶 Taylor 展开后，表达为增量形式的线性高斯-马尔可夫函数模型：

$$\mathbf{z}=\mathbf{Hx}+\boldsymbol{\Delta} \tag{11}$$

若将待估绝对参数记为 $\mathbf{X}$（即 $\mathbf{X}=\mathbf{X}^*+\mathbf{x}$），则模型等价表示为 $\mathbf{Z}=\mathbf{HX}+\boldsymbol{\Delta}$。

设当前历元有效 GPS 双差星数为 m_{DD}，BDS 双差星数为 n_{DD}，双差总星数 $S=m_{DD}+n_{DD}$。观测值总数 $l=4S$（双系统双频双差伪距与载波），待估参数总数 $n=3+2S$，多余观测数（自由度）$r=l-n=2S-3$。

1. 待估参数改正数向量 $\mathbf{x}_{(n\times1)}$： 由流动站三维坐标改正数与各频段双差整周模糊度参数构成：

$$\mathbf{x}=\begin{bmatrix} x \\ y \\ z \\ \nabla\Delta N^{1}_{G,L1}\cdots\nabla\Delta N^{m_{DD}}_{G,L1} \\ \nabla\Delta N^{1}_{G,L2}\cdots\nabla\Delta N^{m_{DD}}_{G,L2} \\ \nabla\Delta N^{1}_{B,B1}\cdots\nabla\Delta N^{n_{DD}}_{B,B1} \\ \nabla\Delta N^{1}_{B,B3}\cdots\nabla\Delta N^{n_{DD}}_{B,B3} \end{bmatrix}_{(3+2S)\times1} = \begin{bmatrix} \mathbf{x}_R \\ \mathbf{N}_{G,L1} \\ \mathbf{N}_{G,L2} \\ \mathbf{N}_{B,B1} \\ \mathbf{N}_{B,B3} \end{bmatrix} \tag{12}$$

其中模糊度的近似值通常取 0，故模糊度改正数即等于模糊度参数本身。

2. 观测向量（OMC 增量向量）$\mathbf{z}_{(l\times1)}$： 定义为实际双差观测值减去近似值计算量 $\mathbf{z}=\mathbf{Z}-\mathbf{F}(\mathbf{X}^{*})$，按系统与频率划分为 8 个子块（总维度 $l\times1=4S\times1$）：

$$\mathbf{z}=\begin{bmatrix} \mathbf{z}_{P,G1} \\ \mathbf{z}_{L,G1} \\ \mathbf{z}_{P,G2} \\ \mathbf{z}_{L,G2} \\ \mathbf{z}_{P,B1} \\ \mathbf{z}_{L,B1} \\ \mathbf{z}_{P,B3} \\ \mathbf{z}_{L,B3} \end{bmatrix}_{(4S)\times1} \tag{13}$$

对于待估双差星 j（参考星为 i），其元素与代码结构体完全对应：

$$z^{j}_{P,f}=(\Delta P^{j}_{f}-\Delta P^{i}_{f})-\rho^{ij*}_{DD}=(\texttt{sdSat[j].dP[f]}-\texttt{sdSat[i].dP[f]})-\rho^{ij*}_{DD} \tag{14}$$

$$z^{j}_{L,f}=(\Delta L^{j}_{f}-\Delta L^{i}_{f})-\rho^{ij*}_{DD}=(\texttt{sdSat[j].dL[f]}-\texttt{sdSat[i].dL[f]})-\rho^{ij*}_{DD} \tag{15}$$

3. 设计矩阵（系数矩阵）$\mathbf{H}_{(l\times n)}$： 按观测类型与系统频率划分，设计矩阵的分块展开形式如下：

$$\mathbf{H}=\begin{bmatrix} \mathbf{H}_{P,G1} \\ \mathbf{H}_{L,G1} \\ \mathbf{H}_{P,G2} \\ \mathbf{H}_{L,G2} \\ \mathbf{H}_{P,B1} \\ \mathbf{H}_{L,B1} \\ \mathbf{H}_{P,B3} \\ \mathbf{H}_{L,B3} \end{bmatrix}_{(4S)\times(3+2S)} = \begin{bmatrix} \mathbf{H}_{X,G} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{H}_{X,G} & \lambda_{G1}\mathbf{I}_{m_{DD}} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{H}_{X,G} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{H}_{X,G} & \mathbf{0} & \lambda_{G2}\mathbf{I}_{m_{DD}} & \mathbf{0} & \mathbf{0} \\ \mathbf{H}_{X,B} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{H}_{X,B} & \mathbf{0} & \mathbf{0} & \lambda_{B1}\mathbf{I}_{n_{DD}} & \mathbf{0} \\ \mathbf{H}_{X,B} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{H}_{X,B} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \lambda_{B3}\mathbf{I}_{n_{DD}} \end{bmatrix} \tag{16}$$

其中，GPS 和 BDS 的流动站位置几何设计子矩阵分别为：

$$\mathbf{H}_{X,G} = \begin{bmatrix} -\Delta l_G^1 & -\Delta m_G^1 & -\Delta n_G^1 \\ -\Delta l_G^2 & -\Delta m_G^2 & -\Delta n_G^2 \\ \vdots & \vdots & \vdots \\ -\Delta l_G^{m_{DD}} & -\Delta m_G^{m_{DD}} & -\Delta n_G^{m_{DD}} \end{bmatrix}_{m_{DD}\times 3} \tag{17}$$

$$\mathbf{H}_{X,B} = \begin{bmatrix} -\Delta l_B^1 & -\Delta m_B^1 & -\Delta n_B^1 \\ -\Delta l_B^2 & -\Delta m_B^2 & -\Delta n_B^2 \\ \vdots & \vdots & \vdots \\ -\Delta l_B^{n_{DD}} & -\Delta m_B^{n_{DD}} & -\Delta n_B^{n_{DD}} \end{bmatrix}_{n_{DD}\times 3} \tag{18}$$

设计矩阵的结构特点为：

- **前 3 列**：对应流动站坐标改正数 $[x, y, z]^T$，所有观测类型方程对应的系数均为视线方向余弦差负值 $[-\Delta l, -\Delta m, -\Delta n]$；
- **伪距观测块**：在所有模糊度参数列上的系数均为 0；
- **载波观测块**：仅在当前卫星、当前频点对应的双差模糊度参数列上填入载波波长 λ_f，其余列均为 0。

3.2.4 四、随机模型构建与闭合解析权矩阵

1. 误差统计特性与协方差矩阵： 随机误差向量服从零均值正态分布：$\boldsymbol{\Delta} \sim N(\mathbf{0}, \mathbf{D})$，即 $E(\boldsymbol{\Delta}) = \mathbf{0}$，$\mathrm{var}(\boldsymbol{\Delta}) = \mathbf{D}$。设基准站与流动站非差观测误差独立同分布，方差为 σ^2。站间单差观测值方差为 $\mathrm{var}(\Delta L) = \mathrm{var}(L_R) + \mathrm{var}(L_B) = 2\sigma^2$。不同卫星的单差误差互不相关。但在构成星间双差时，同一频点的所有非参考星均减去了同一颗参考星 i：

$$\begin{bmatrix} \nabla\Delta L^1 \\ \nabla\Delta L^2 \\ \vdots \\ \nabla\Delta L^{n_{DD}} \end{bmatrix} = \begin{bmatrix} -1 & 1 & 0 & \cdots & 0 \\ -1 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ -1 & 0 & 0 & \cdots & 1 \end{bmatrix}_{n_{DD}\times(n_{DD}+1)} \begin{bmatrix} \Delta L^i \\ \Delta L^1 \\ \Delta L^2 \\ \vdots \\ \Delta L^{n_{DD}} \end{bmatrix}$$

记差分算子矩阵为 $\mathbf{C}$，由协方差传播律 $\mathbf{D}_{DD} = \mathbf{C}(2\sigma^2\mathbf{I})\mathbf{C}^T$，求得同一频段 n_{DD} 个双差观测值的协方差矩阵为：

$$\mathbf{D}_{DD} = 2\sigma^2 \begin{bmatrix} 2 & 1 & \cdots & 1 \\ 1 & 2 & \cdots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \cdots & 2 \end{bmatrix}_{n_{DD}\times n_{DD}} \tag{19}$$

2. 权矩阵解析逆求法： 定义权矩阵与协方差矩阵、单位权方差 σ_0^2 的关系为：

$$\mathbf{W} = \sigma_0^2 \mathbf{D}^{-1} \tag{20}$$

取单位权方差 $\sigma_0^2 = 1$，则子块权矩阵 $\mathbf{W}_{sub} = \mathbf{D}_{DD}^{-1}$。对特殊对称矩阵 $\mathbf{D}_{DD}$ 严格求逆，可直接写出闭合解析权阵，无需数值求逆：

$$\mathbf{W}_{sub} = \mathbf{D}_{DD}^{-1} = \frac{1}{2\sigma^2(n_{DD}+1)} \begin{bmatrix} n_{DD} & -1 & \cdots & -1 \\ -1 & n_{DD} & \cdots & -1 \\ \vdots & \vdots & \ddots & \vdots \\ -1 & -1 & \cdots & n_{DD} \end{bmatrix}_{n_{DD}\times n_{DD}} \tag{21}$$

即矩阵主对角线元素为 $\frac{n_{DD}}{2\sigma^2(n_{DD}+1)}$，非对角线元素统一为 $\frac{-1}{2\sigma^2(n_{DD}+1)}$。

根据常用 GNSS 先验噪声设置：

- 伪距先验测距误差：$\sigma_P = 0.3\,\mathrm{m}$（$\sigma_P^2 = 0.09\,\mathrm{m}^2$）
- 载波相位测距误差：$\sigma_L = 0.01\,\mathrm{m}$（$\sigma_L^2 = 0.0001\,\mathrm{m}^2$）

由于不同系统、不同频率、不同观测类型之间误差不相关，全局权矩阵 $\mathbf{W}$ 为准对角分块矩阵：

$$\begin{aligned} \mathbf{W} = \mathrm{diag}\Big(&\mathbf{W}_{P,G1},\ \mathbf{W}_{L,G1},\ \mathbf{W}_{P,G2},\ \mathbf{W}_{L,G2},\\ &\mathbf{W}_{P,B1},\ \mathbf{W}_{L,B1},\ \mathbf{W}_{P,B3},\ \mathbf{W}_{L,B3}\Big)_{(4S)\times(4S)} \end{aligned} \tag{22}$$

对应的协因数矩阵定义为 $\mathbf{Q} = \mathbf{W}^{-1}$。

3.2.5 五、最小二乘估计求解与精度评定流程

在 `RTKFloat` 中按以下标准流程执行解算：

(1) **初值初始化**：基站设定为已知真实坐标 $\mathbf{X}_B$；流动站调用 SPP 获取当前历元初始坐标 $\mathbf{X}_R^*$。

(2) **双差基准星与星数统计**：调用 `DetRefSat`，获取 GPS 和 BDS 参考星索引及各频点双差星数。检查方程可解性（$S \geq 3$ 且 $l > n$）。

(3) **计算基站几何距离**：计算基站到所有可用卫星的几何距离 d_B^s，基站坐标固定，外层仅需计算一次。

(4) **开始迭代循环**（最大迭代次数设为 10 次）：

(5) **计算流动站到参考星的几何距离与方向余弦**：根据当前流动站估计值 $\mathbf{X}_R^*$，计算流动站到 GPS 和 BDS 参考星的几何距离 d_R^{i*} 与单位方向余弦 (l^i, m^i, n^i)。

(6) **遍历有效单差观测值构建 H 与 z**：

- 跳过参考星或周跳无效星（`Valid == 0`）；
- 计算流动站到待估星的距离 d_R^{j*} 与方向余弦 (l^j, m^j, n^j)；
- 计算方向余弦差 $\Delta l^{ij}, \Delta m^{ij}, \Delta n^{ij}$ 与几何初值 $\rho_{DD}^{ij*} = (d_R^{j*} - d_R^{i*}) - (d_B^j - d_B^i)$；
- 观测残差：$z_{P,f}^j = (\Delta P_f^j - \Delta P_f^i) - \rho_{DD}^{ij*}$，$z_{L,f}^j = (\Delta L_f^j - \Delta L_f^i) - \rho_{DD}^{ij*}$；
- 填充设计矩阵 $\mathbf{H}$：坐标列填 $-\Delta l^{ij}, -\Delta m^{ij}, -\Delta n^{ij}$，载波方程在对应模糊度列填 λ_f。

(7) **构建双差权矩阵 W**：利用闭合解析求逆公式分别组装 GPS 与 BDS 伪距与载波各子块权矩阵。

(8) **最小二乘法方程解算**：法矩阵为 $\mathbf{H}^T\mathbf{W}\mathbf{H}$，法方程为：

$$\mathbf{H}^T\mathbf{W}\mathbf{H}\hat{\mathbf{x}} = \mathbf{H}^T\mathbf{W}\mathbf{z}$$

解算参数增量与协因数阵：

$$\hat{\mathbf{x}} = (\mathbf{H}^T\mathbf{W}\mathbf{H})^{-1}\mathbf{H}^T\mathbf{W}\mathbf{z}, \quad \mathbf{Q}_{\hat{\mathbf{x}}} = (\mathbf{H}^T\mathbf{W}\mathbf{H})^{-1}$$

(9) **更新流动站位置参数**：

$$\mathbf{X}_R^* = \mathbf{X}_R^* + \hat{\mathbf{x}}(0..2)$$

(10) **收敛性判定**：计算坐标改正数模长 $\delta = \|\hat{\mathbf{x}}(0..2)\|_2 = \sqrt{\hat{x}^2 + \hat{y}^2 + \hat{z}^2}$。若 $\delta > 1.0 \times 10^{-4}\,\mathrm{m}$ 且迭代未超限，返回第 5 步进行下一次迭代；若 $\delta \le 1.0 \times 10^{-4}\,\mathrm{m}$，判定收敛，迭代结束。

(11) **输出浮点解与精度评定**：

- 基线向量：$\mathbf{dPos} = \mathbf{X}_R - \mathbf{X}_B$；
- 提取双差浮点模糊度向量 $\hat{\mathbf{a}} = \hat{\mathbf{x}}[3 \ldots 3+2S-1]$ 与模糊度协因数阵 $\mathbf{Q}_{\hat{\mathbf{a}}\hat{\mathbf{a}}} = \mathbf{Q}_{\hat{\mathbf{x}}}[3 \ldots 3+2S-1, 3 \ldots 3+2S-1]$，保存至 `Raw->DDObs` 为 LAMBDA 算法提供输入；
- 残差向量 $\mathbf{v} = \mathbf{z} - \mathbf{H}\hat{\mathbf{x}}$；
- 验后单位权中误差：

$$\hat{\sigma}_0 = \sqrt{\frac{\mathbf{v}^T\mathbf{W}\mathbf{v}}{l-n}} = \sqrt{\frac{\mathbf{v}^T\mathbf{W}\mathbf{v}}{4S-(3+2S)}}$$

- 验后参数协方差矩阵：$\hat{\mathbf{D}}_{\hat{\mathbf{x}}} = \hat{\sigma}_0^2\mathbf{Q}_{\hat{\mathbf{x}}}$；
- 基线分量中误差：$m_X = \hat{\sigma}_0\sqrt{Q_{00}}$，$m_Y = \hat{\sigma}_0\sqrt{Q_{11}}$，$m_Z = \hat{\sigma}_0\sqrt{Q_{22}}$。

3.2.6 六、RTKFloat 核心算法

根据上述严密推导与算法流程，给出在 FormSDEpochObs.cpp 中使用 Eigen 库实现 RTKFloat 的代码蓝图，变量命名与矩阵构建完全对应上述符号：

```
#include <Eigen/Dense>
using namespace Eigen;

// 构建单频段 nDD 颗卫星的双差权阵子块 (闭合解析解 W_sub = D_DD^{-1})
MatrixXd BuildSubWeightMatrix(int nDD, double sigma) {
    MatrixXd W_sub(nDD, nDD);
    double sigma2 = sigma * sigma;
    double factor = 1.0 / (2.0 * sigma2 * (nDD + 1));
    for (int r = 0; r < nDD; ++r) {
        for (int c = 0; c < nDD; ++c) {
            W_sub(r, c) = (r == c ? (double)nDD : -1.0) * factor;
        }
    }
    return W_sub;
}

// RTK 双差浮点解算主函数
bool RTKFloat(RAWDAT* Raw, PPRESULT* Base, PPRESULT* Rov) {
    if (!Raw || !Base || !Rov) return false;

    // 1. 基站已知真实坐标 X_B
    Vector3d X_B(Raw->BasEpk.Pos[0], Raw->BasEpk.Pos[1], Raw->BasEpk.Pos[2]);

    // 2. 流动站 SPP 单点定位获取初始近似坐标 X_R_star
    DetectOutlier(&Raw->RovEpk);
    if (!SPP(&Raw->RovEpk, Raw, Rov)) return false;
    Vector3d X_R_star(Rov->Position[0], Rov->Position[1], Rov->Position[2]);

    // 3. 选取参考星并统计双差星数
    DetRefSat(&Raw->BasEpk, &Raw->RovEpk, &Raw->SdObs, &Raw->DDObs);
    int m_DD = Raw->DDObs.DDSatNum[0]; // GPS 双差星数
```

```
    int n_DD = Raw->DDObs.DDSatNum[1]; // BDS 双差星数
    int S    = m_DD + n_DD;            // 双差总星数
    int n    = 3 + 2 * S;              // 待估参数总数 (3坐标改正数 + 2S双差模糊度)
    int l    = 4 * S;                  // 双差观测方程总数 (双系统双频双差)

    if (l <= n || S < 3) return false; // 多余观测数 r = l - n 必须大于 0

    // 获取两系统参考星在单差结构体中的索引
    int ref_idx[2] = { Raw->DDObs.RefPos[0], Raw->DDObs.RefPos[1] };

    // 4. 计算基站到各卫星的固定几何距离 d_B
    std::vector<double> d_B(Raw->SdObs.SatNum, 0.0);
    for (int i = 0; i < Raw->SdObs.SatNum; ++i) {
        int nBas = Raw->SdObs.SdSatObs[i].nBas;
        const double* sat_pos = Raw->BasEpk.SatPVT[nBas].SatPos;
        double dx = sat_pos[0] - X_B[0], dy = sat_pos[1] - X_B[1], dz = sat_pos[2] - X_B[2];
        d_B[i] = sqrt(dx * dx + dy * dy + dz * dz);
    }

    // 5. 组装全局双差权矩阵 W (准对角分块矩阵)
    MatrixXd W = MatrixXd::Zero(l, l);
    double sigP = 0.3, sigL = 0.01;
    if (m_DD > 0) {
        MatrixXd W_PG1 = BuildSubWeightMatrix(m_DD, sigP);
        MatrixXd W_LG1 = BuildSubWeightMatrix(m_DD, sigL);
        MatrixXd W_PG2 = BuildSubWeightMatrix(m_DD, sigP);
        MatrixXd W_LG2 = BuildSubWeightMatrix(m_DD, sigL);
        W.block(0 * m_DD, 0 * m_DD, m_DD, m_DD) = W_PG1;
        W.block(1 * m_DD, 1 * m_DD, m_DD, m_DD) = W_LG1;
        W.block(2 * m_DD, 2 * m_DD, m_DD, m_DD) = W_PG2;
        W.block(3 * m_DD, 3 * m_DD, m_DD, m_DD) = W_LG2;
    }
    if (n_DD > 0) {
        int offset = 4 * m_DD;
        MatrixXd W_PB1 = BuildSubWeightMatrix(n_DD, sigP);
```

```
67          MatrixXd W_LB1 = BuildSubWeightMatrix(n_DD, sigL);
68          MatrixXd W_PB3 = BuildSubWeightMatrix(n_DD, sigP);
69          MatrixXd W_LB3 = BuildSubWeightMatrix(n_DD, sigL);
70          W.block(offset + 0 * n_DD, offset + 0 * n_DD, n_DD, n_DD
                ) = W_PB1;
71          W.block(offset + 1 * n_DD, offset + 1 * n_DD, n_DD, n_DD
                ) = W_LB1;
72          W.block(offset + 2 * n_DD, offset + 2 * n_DD, n_DD, n_DD
                ) = W_PB3;
73          W.block(offset + 3 * n_DD, offset + 3 * n_DD, n_DD, n_DD
                ) = W_LB3;
74      }
75
76      // 6. 高斯-牛顿最小二乘迭代循环
77      bool converged = false;
78      MatrixXd Qxx(n, n);
79      VectorXd x_hat(n);
80
81      for (int iter = 0; iter < 10; ++iter) {
82          // A. 计算流动站到参考星的几何距离与方向余弦
83          double d_R_ref[2] = {0}, l_ref[2] = {0}, m_ref[2] = {0},
                n_ref[2] = {0};
84          for (int sys = 0; sys < 2; ++sys) {
85              if (ref_idx[sys] == -1) continue;
86              int nRov = Raw->SdObs.SdSatObs[ref_idx[sys]].nRov;
87              const double* spos = Raw->RovEpk.SatPVT[nRov].SatPos
                    ;
88              double dx = spos[0] - X_R_star[0], dy = spos[1] -
                    X_R_star[1], dz = spos[2] - X_R_star[2];
89              d_R_ref[sys] = sqrt(dx * dx + dy * dy + dz * dz);
90              l_ref[sys] = dx / d_R_ref[sys]; // (X^s - X_R_star)
                    / d
91              m_ref[sys] = dy / d_R_ref[sys]; // (Y^s - Y_R_star)
                    / d
92              n_ref[sys] = dz / d_R_ref[sys]; // (Z^s - Z_R_star)
                    / d
93          }
94
95          // B. 初始化设计矩阵 H 与 OMC 观测向量 z (对应模型 z = H
                * x + Delta)
```

```
96          MatrixXd H = MatrixXd::Zero(l, n);
97          VectorXd z = VectorXd::Zero(l);
98
99          int k_G = 0, k_C = 0;
100         for (int i = 0; i < Raw->SdObs.SatNum; ++i) {
101             const SDSATOBS& sd = Raw->SdObs.SdSatObs[i];
102             if (sd.Valid == 0) continue;
103             int sys = (sd.System == GPS) ? 0 : 1;
104             if (i == ref_idx[sys]) continue; // 跳过参考星
105
106             int nRov = sd.nRov;
107             const double* spos = Raw->RovEpk.SatPVT[nRov].SatPos
                    ;
108             double dx = spos[0] - X_R_star[0], dy = spos[1] -
                    X_R_star[1], dz = spos[2] - X_R_star[2];
109             double d_R_j = sqrt(dx * dx + dy * dy + dz * dz);
110             double l_j = dx / d_R_j, m_j = dy / d_R_j, n_j = dz
                    / d_R_j;
111
112             // 方向余弦差 (待估星 - 参考星)
113             double dl = l_j - l_ref[sys];
114             double dm = m_j - m_ref[sys];
115             double dn = n_j - n_ref[sys];
116
117             // 双差几何距离近似计算量 rho_DD_star
118             double rho_DD_star = (d_R_j - d_R_ref[sys]) - (d_B[i
                    ] - d_B[ref_idx[sys]]);
119
120             const SDSATOBS& sd_ref = Raw->SdObs.SdSatObs[ref_idx
                    [sys]];
121             if (sd.System == GPS) {
122                 // GPS L1 伪距与载波
123                 z(0 * m_DD + k_G) = (sd.dP[0] - sd_ref.dP[0]) -
                        rho_DD_star;
124                 H.block<1, 3>(0 * m_DD + k_G, 0) << -dl, -dm, -
                        dn;
125
126                 z(1 * m_DD + k_G) = (sd.dL[0] - sd_ref.dL[0]) -
                        rho_DD_star;
127                 H.block<1, 3>(1 * m_DD + k_G, 0) << -dl, -dm, -
```

```
                    dn;
128                 H(1 * m_DD + k_G, 3 + k_G) = WL1_GPS;
129
130                 // GPS L2 伪距与载波
131                 z(2 * m_DD + k_G) = (sd.dP[1] - sd_ref.dP[1]) -
                        rho_DD_star;
132                 H.block<1, 3>(2 * m_DD + k_G, 0) << -dl, -dm, -
                        dn;
133
134                 z(3 * m_DD + k_G) = (sd.dL[1] - sd_ref.dL[1]) -
                        rho_DD_star;
135                 H.block<1, 3>(3 * m_DD + k_G, 0) << -dl, -dm, -
                        dn;
136                 H(3 * m_DD + k_G, 3 + m_DD + k_G) = WL2_GPS;
137
138                 k_G++;
139             } else if (sd.System == BDS) {
140                 int off = 4 * m_DD;
141                 // BDS B1 伪距与载波
142                 z(off + 0 * n_DD + k_C) = (sd.dP[0] - sd_ref.dP
                        [0]) - rho_DD_star;
143                 H.block<1, 3>(off + 0 * n_DD + k_C, 0) << -dl, -
                        dm, -dn;
144
145                 z(off + 1 * n_DD + k_C) = (sd.dL[0] - sd_ref.dL
                        [0]) - rho_DD_star;
146                 H.block<1, 3>(off + 1 * n_DD + k_C, 0) << -dl, -
                        dm, -dn;
147                 H(off + 1 * n_DD + k_C, 3 + 2 * m_DD + k_C) =
                        WL1_BDS;
148
149                 // BDS B3 伪距与载波
150                 z(off + 2 * n_DD + k_C) = (sd.dP[1] - sd_ref.dP
                        [1]) - rho_DD_star;
151                 H.block<1, 3>(off + 2 * n_DD + k_C, 0) << -dl, -
                        dm, -dn;
152
153                 z(off + 3 * n_DD + k_C) = (sd.dL[1] - sd_ref.dL
                        [1]) - rho_DD_star;
154                 H.block<1, 3>(off + 3 * n_DD + k_C, 0) << -dl, -
```

```
                    dm, -dn;
155                 H(off + 3 * n_DD + k_C, 3 + 2 * m_DD + n_DD +
                        k_C) = WL3_BDS;
156
157                 k_C++;
158             }
159         }
160
161         // C. 最小二乘求解法方程: (H^T * W * H) * x_hat = H^T *
                W * z
162         MatrixXd HtWH = H.transpose() * W * H;
163         VectorXd HtWz = H.transpose() * W * z;
164         Qxx = HtWH.inverse();
165         x_hat = Qxx * HtWz;
166
167         // D. 更新流动站近似坐标: X_R_star = X_R_star + x_hat
                [0..2]
168         X_R_star[0] += x_hat(0);
169         X_R_star[1] += x_hat(1);
170         X_R_star[2] += x_hat(2);
171
172         // E. 收敛判定 (坐标改正数模长 < 0.1 mm)
173         double dx_norm = x_hat.head<3>().norm();
174         if (dx_norm < 1.0e-4) {
175             converged = true;
176             break;
177         }
178     }
179
180     if (!converged) return false;
181
182     // 7. 解算成功, 保存基线结果与精度指标
183     Rov->Position[0] = X_R_star[0];
184     Rov->Position[1] = X_R_star[1];
185     Rov->Position[2] = X_R_star[2];
186
187     Raw->DDObs.dPos[0] = X_R_star[0] - X_B[0];
188     Raw->DDObs.dPos[1] = X_R_star[1] - X_B[1];
189     Raw->DDObs.dPos[2] = X_R_star[2] - X_B[2];
190
```

```
    // 残差向量 v = z - H * x_hat 与验后单位权中误差 sigma0_hat
    // VectorXd v = z - H * x_hat;
    // double sigma0_2 = (v.transpose() * W * v)(0) / (l - n);
    // double sigma0 = sqrt(sigma0_2);

    // 提取双差浮点模糊度 a_hat = x_hat[3 ... 3 + 2S - 1]
    // 提取模糊度协因数矩阵 Qaa = Qxx.block(3, 3, 2 * S, 2 * S)
    // 保存至 Raw->DDObs 用于后续 LAMBDA 算法进行整数模糊度固定

    return true;
}
```

3.3　issue#1 流动站高程异常漂移导致 4097 历元对流层模型崩溃

使用全量短基线实测数据（流动站 oem719-202510271500-rover.bin，基站 base.bin，共 10075 历元），进行双差相对定位浮点解（RTKFloat）测试：

预期结果：除首历元外，其余 10074 个历元全部正常解出双差浮点解。

实际结果：大部分历元能解出，但在第 4097 ∼ 4122 历元（TOW 116707 ∼ 116732）出现连续 26 个历元解算失败（RTK Float FAILED），随后在第 4125 历元连续失败 8 次，第 4136 历元连续失败 21 次。

```
Sync epoch 4096: TOW 116706.000, RTK Float OK, Ref(G16/C39), DD 24 sats...
Sync epoch 4097: TOW 116707.000, RTK Float FAILED (Bas 24 sats, Rov 31 sats, DD 24 sats)
Sync epoch 4098: TOW 116708.000, RTK Float FAILED (Bas 24 sats, Rov 31 sats, DD 24 sats)
...
Sync epoch 4122: TOW 116732.000, RTK Float FAILED (Bas 24 sats, Rov 31 sats, DD 24 sats)
Sync epoch 4123: TOW 116733.000, RTK Float OK, Ref(G16/C39), DD 24 sats...
```

查明原因：

(1) 打印第 4097 历元最小二乘平差矩阵，发现设计矩阵 **H** 与观测余差向量 **z** 的元素全部变成了 NaN，导致平差法方程无解。

(2) 逐项排查观测余差，发现是对流层延迟计算输出了 NaN。在 detect.cpp 中：

```
double base = 1.0 - 0.0000226 * (H - HIGHT0);
double p = PRESSURE0 * pow(base, 5.225);
```

当高程 $H > 44247.78\,\mathrm{m}$ 时，base 变为负数。原代码仅在外部判断了 H > 50000.0，当 $H \in (44248, 50000)$ 时对负数求非整数次幂 pow(base, 5.225)，在实数域无解，返回 NaN。

(3) 流动站高程为什么会跑到 44 km？回溯发现：

第 4 历元（TOW 112614.0）北斗星历首次解析出来后，北斗 C56 卫星进入 SPP 计算。检查原始星历发现 C56 广播星历中标记 `SVHealth = 1`（卫星不健康），且参考时刻 $t_{oe} = 284400\,\mathrm{s}$ 距当前时刻相差近 48 小时。原 `SatPVT.cpp` 中没有检查 `eph->SVHealth` 和星历龄期，导致不健康卫星被带入 SPP 计算，伪距残差高达 1311 km，直接把第 4 历元流动站高程拉偏到 $-286337.8\,\mathrm{m}$。

(4) 流动站 SPP 以前一历元作为初值递推，随卫星运动，这个错误的高程在空中缓慢漂移（历元 4 时 $-286\,\mathrm{km} \rightarrow$ 历元 2000 时 $-107\,\mathrm{km} \rightarrow$ 历元 3500 时 $+23\,\mathrm{km} \rightarrow$ 历元 4096 时 $+44246.8\,\mathrm{m}$）。到第 4097 历元高程升至 44269.6 m，刚好越过 44247.8 m 门限，触发负数幂产生 `NaN`。

(5) 为什么会连续成块失败？
流动站数据中没有 `BESTPOS` 报文（全文件扫描为 0 帧），流动站初值完全由单点定位维护。SPP 失败后未清零 `Epoch->Pos`，而原初值保护逻辑只检查了 `Pos[0] == 0.0`，导致后续历元死锁继承错误的 44 km 初值，连续解算失败。

3.4 fix of issue#1 增加卫星健康度检查与对流层模型边界保护

修复思路：

(1) 在 `SatPVT.cpp` 的 `CompSatClkOff`、`CompGPSSatPVT`、`CompBDSSatPVT` 中加入 `eph->SVHealth != 0` 和星历有效龄期（4 小时）检查，不健康或过期卫星直接拒绝计算，从源头剔除 C56。

(2) 在 `detect.cpp` 的 `Hopfield` 中增加底数非负检查，将高程有效上限设为 40,000 m，底数 ≤ 0 时返回 0.0。

(3) 在 `FormSDEpochObs.cpp` 中增加几何判断：偏离基站超过 20 km 时强制重置为基站坐标，打破错误初值的死锁继承。

关键代码：

```
// 1. SatPVT.cpp: 钟差与轨道计算前拦截不健康卫星与过期星历
bool CompSatClkOff(...) {
    if (eph == NULL || eph->PRN == 0) return false;
    if (eph->SVHealth != 0) return false; // 卫星不健康，拒绝计
       算
    ...
}
bool CompBDSSatPVT(...) {
    if (eph == NULL || eph->PRN == 0) return false;
    if (eph->SVHealth != 0) return false;
```

```
    ...
    double tk = GetDiffTime(&t_bds, &eph->TOE);
    if (fabs(tk) > 14400.0) return false; // 星历龄期超过4小时拒绝计算
    ...
}

// 2. detect.cpp: Hopfield 模型增加底数截断
double Hopfield(double H, double Elev) {
    if (H < -1000.0 || H > 40000.0) return 0.0;
    double base = 1.0 - 0.0000226 * (H - HIGHT0);
    if (base <= 0.0) return 0.0; // 避免负数底数导致 pow 产生NaN
    ...
}

// 3. FormSDEpochObs.cpp: 偏离基站过远时强制复位
if (Raw->RovEpk.Pos[0] == 0.0 || (Vector3d(Raw->RovEpk.Pos) - X_B).norm() > 20000.0) {
    Raw->RovEpk.Pos[0] = Base->Position[0];
    Raw->RovEpk.Pos[1] = Base->Position[1];
    Raw->RovEpk.Pos[2] = Base->Position[2];
}
```

验证结果：全量 10075 历元重新运行，除第 1 历元无双差外，其余 10074 历元全部正常解出，成功率 100.0%。第 4 历元高程稳定在 38.8 m，原 4097～4122 历元连续失败现象彻底消除。

```
Sync epoch 1: TOW 112611.000, RTK Float FAILED (Bas 24 sats, Rov 31 sats, DD 0 sats)
Sync epoch 2: TOW 112612.000, RTK Float OK, Ref(G26/C-1), DD 8 sats(G8/C0), dPos: [-474.4614, 675.1378, -1358.5406], Len: 1589.5147 m, Sig0: 1.4388 m
...
Sync epoch 4096: TOW 116706.000, RTK Float OK, Ref(G16/C39), DD 24 sats(G8/C16), dPos: [-474.4604, 675.0450, -1357.8805], Len: 1588.9109 m, Sig0: 1.6687 m
Sync epoch 4097: TOW 116707.000, RTK Float OK, Ref(G16/C39), DD 24 sats(G8/C16), dPos: [-474.4602, 675.0451, -1357.8804], Len: 1588.9108 m, Sig0: 1.6687 m
...
Sync epoch 10075: TOW 122685.000, RTK Float OK, Ref(G08/C06), DD 21 sats(G7/C14), dPos: [-474.4280, 674.6565, -1358.3021], Len: 1589.0966 m, Sig0: 1.1596 m

RTK Processing Summary:
Total Synced Epochs: 10075
RTK Float Success:   10074 (100.0%)
Results saved to ./result.txt
```

4 2026-09-23 星期三

4.1 `feat#3` 整周模糊度固定与 LAMBDA 算法原理

在完成站星双差浮点解（`RTKFloat`）计算后，求得的未知数向量中包含流动站坐标改正数 $\hat{\mathbf{x}}_R$ 与双差模糊度浮点解 $\hat{\mathbf{a}}$ 及其协方差矩阵 $\mathbf{Q}_{\hat{\mathbf{a}}\hat{\mathbf{a}}}$。由于双差观测值中已彻底消除了接收机与卫星的初始相位与钟差偏差，双差模糊度理论上具有严格的整数特性（$\check{\mathbf{a}} \in \mathbb{Z}^n$）。将浮点模糊度准确固定为正确整数是实现厘米级至毫米级高精度 RTK 定位的关键步骤。

4.1.1 一、整数最小二乘准则 (ILS)

整周模糊度固定问题在数学上归结为有约束的整数最小二乘（Integer Least-Squares, ILS）优化问题：

$$\min_{\check{\mathbf{a}}\in\mathbb{Z}^n} \|\hat{\mathbf{a}} - \check{\mathbf{a}}\|^2_{\mathbf{Q}^{-1}_{\hat{\mathbf{a}}\hat{\mathbf{a}}}} = \min_{\check{\mathbf{a}}\in\mathbb{Z}^n} (\hat{\mathbf{a}} - \check{\mathbf{a}})^T \mathbf{Q}^{-1}_{\hat{\mathbf{a}}\hat{\mathbf{a}}} (\hat{\mathbf{a}} - \check{\mathbf{a}}) \tag{23}$$

该优化问题的几何意义是在以浮点解 $\hat{\mathbf{a}}$ 为中心、由协方差阵 $\mathbf{Q}_{\hat{\mathbf{a}}\hat{\mathbf{a}}}$ 定义的超椭球内寻找距离最近的整值网格点 $\check{\mathbf{a}}$。

在 GNSS 双差观测模型中，由于共用参考星以及几何视线方向的相似性，双差模糊度参数之间存在极强的空间相关性（相关系数常高达 0.99 以上）。这导致误差椭球极度扁平且狭长，若采用常规的逐维舍入（Rounding）或直接在原始空间进行超椭球网格点搜索，搜索空间极其巨大，计算复杂度呈指数级爆炸，极易陷入长循环或搜索失败。

4.1.2 二、LAMBDA 降相关与整数 Z 变换

荷兰代尔夫特理工大学 Teunissen 教授提出的 LAMBDA（Least-squares AMBiguity Decorrelation Adjustment）算法，通过构造一个可逆的整数单模变换矩阵 $\mathbf{Z}$（Unimodular Matrix），将强相关的原始模糊度向量 $\mathbf{a}$ 投影到一个弱相关的虚拟空间 $\mathbf{z}$：

$$\mathbf{z} = \mathbf{Z}^T\mathbf{a}, \quad \hat{\mathbf{z}} = \mathbf{Z}^T\hat{\mathbf{a}}, \quad \mathbf{Q}_{\hat{\mathbf{z}}\hat{\mathbf{z}}} = \mathbf{Z}^T\mathbf{Q}_{\hat{\mathbf{a}}\hat{\mathbf{a}}}\mathbf{Z} \tag{24}$$

整数单模矩阵 $\mathbf{Z}$ 满足：

(1) 所有元素均为整数：$Z_{ij} \in \mathbb{Z}$；

(2) 行列式绝对值为 1：$|\det(\mathbf{Z})| = 1$；

(3) 逆矩阵 $\mathbf{Z}^{-1}$ 同样为全整数矩阵，保持整数网格映射的一一对应性与体积不变性。

LAMBDA 算法通过迭代执行以下两个交替步骤完成降相关过程：

(1) **整数高斯变换（Gauss Transformation）**：对协方差矩阵的对角线上三角分解矩阵 $\mathbf{L}$ 的非对角元素执行整数消元，消除维度间的协方差相关性：

$$\mathbf{L}_{k,j} \leftarrow \mathbf{L}_{k,j} - \mu \mathbf{L}_{k,i}, \quad \mu = \text{round}(\mathbf{L}_{i,j})$$

(2) **列置换与对角元素重排（Permutation）**：当相邻维度的条件方差违背降序规则时，交换变量序号并重新调整条件方差，促使搜索椭球的主轴尽量平行于坐标轴且各轴长度相近，将扁平狭长的倾斜超椭球“推挤”为接近正球面的形状。

4.1.3　三、条件方差分解与收缩椭球搜索 (MLAMBDA)

对降相关后的协因数阵进行 LDL^T 分解（代码中对应 `LD()`）：

$$\mathbf{Q}_{\hat{z}\hat{z}} = \mathbf{L}^T \mathbf{D} \mathbf{L} \tag{25}$$

其中 $\mathbf{L}$ 为单位下三角矩阵，$\mathbf{D} = \text{diag}(d_1, d_2, \ldots, d_n)$ 为各维度的条件方差。

根据条件误差平方和的链式展开，目标函数可分解为各维度的单变量级联求和：

$$(\hat{\mathbf{z}} - \mathbf{z})^T \mathbf{Q}_{\hat{z}\hat{z}}^{-1} (\hat{\mathbf{z}} - \mathbf{z}) = \sum_{k=1}^{n} \frac{(\hat{z}_{k|k+1\ldots n} - z_k)^2}{d_k} \leq \chi^2 \tag{26}$$

代码采用改进的 MLAMBDA 搜索算法（`search()`）：从最高维度 $k = n$ 开始自上而下确定各维度的整数搜索边界区间，并在每找到更优解时动态收缩阈值 χ^2，从而能在微秒级极速搜索出残差最小的最优整数候选向量 $\check{\mathbf{z}}_1$ 及次优整数向量 $\check{\mathbf{z}}_2$。最后通过逆变换方程 $\mathbf{Z}^T \check{\mathbf{a}} = \check{\mathbf{z}}$ 恢复出原始空间的整数模糊度。

4.2　feat#4 RTKLIB 开源算法的解耦与工程适配改造

4.2.1　一、开源来源与接口对齐

本项目模糊度解算模块的数学核心引入了开源著名 GNSS 软件 RTKLIB（v2.4.3）的 `lambda.c` 实现。在本次工程实践中，为了完全适配武大测绘学院王老师在 `RTK_Structs.h`（第 410 ～ 416 行）中预留的 6 个标准接口：

```
// RTK_Structs.h 预置的 LAMBDA 算法接口
int LD(int n, const double *Q, double *L, double *D);
void gauss(int n, double *L, double *Z, int i, int j);
void perm(int n, double *L, double *D, int j, double del, double
    *Z);
void reduction(int n, double *L, double *D, double *Z);
int search(int n, int m, const double *L, const double *D,
           const double *zs, double *zn, double *s);
```

```
int lambda(int n, int m, const double *a, const double *Q,
           double *F, double *s);
```

我们不能直接将整个 RTKLIB 庞大的工程代码库引入，必须将 lambda.c 进行原子化剥离与本地工程化重构，创建独立的 lambda.cpp。

4.2.2 二、消除外部 BLAS/LAPACK 与动态库依赖

原版 RTKLIB 的 lambda.c 并非完全自包含的单一文件，其底层深度依赖 RTKLIB 核心工具库 rtkcmn.c 以及外部科学计算库：

(1) **原版 LAPACK 依赖缺陷**：原版在将 z 空间最优整数解 **E** 逆变换回原模糊度空间 **F** 时，调用了内部函数 solve("T", Z, E, n, m, F)。该函数在底层的实现是直接调用经典 Fortran/LAPACK 库的 dgetrf_（LU 分解）和 dgetrs_（线性方程求解）符号。若直接在 Windows MinGW 环境下编译，会直接触发大量找不到符号的链接错误（undefined reference to 'dgetrf_'），需要额外配置庞大的 OpenBLAS 或 MKL 动态链接库。

(2) **基于 Eigen 库的轻量化求解改造**：
由于本项目在 SPP.cpp 与 FormSDEpochObs.cpp 中已全工程引入了现代 C++ 高性能头文件矩阵库 <Eigen/Dense>，我们在 lambda.cpp 内部直接利用 Eigen 的 partialPivLu().solve() 实现了轻量级替代函数：

```
// 矩阵线性方程组求解辅助: 求解 Z^T * F = E 即 F = Z^{-T} * E
// 利用工程已有的 Eigen 求解, 无需外部 BLAS/LAPACK 库
static int solve_mat(const char *tr, const double *A, const double *Y,
                     int n, int m, double *X)
{
    Eigen::Map<const Eigen::MatrixXd> matA(A, n, n);
    Eigen::Map<const Eigen::MatrixXd> matY(Y, n, m);
    Eigen::Map<Eigen::MatrixXd> matX(X, n, m);
    if (tr[0] == 'T' || tr[0] == 't') {
        matX = matA.transpose().partialPivLu().solve(matY);
    } else {
        matX = matA.partialPivLu().solve(matY);
    }
    return 0;
}
```

仅通过 10 行高效代码，就彻底切断了对外部 Fortran/LAPACK 动态库的依赖，使工程保持纯粹的 MinGW 原生编译特性。

4.2.3　三、矩阵运算与内存管理自治化

除了消除 LAPACK 依赖外，我们对原版调用 RTKLIB 全局工具函数的部分也进行了自治化隔离：

(1) **矩阵相乘解耦**：原版计算正变换 $\hat{\mathbf{z}} = \mathbf{Z}^T\hat{\mathbf{a}}$ 时调用了 `matmul("TN", ...)`。在 `lambda.cpp` 中直接使用 Eigen 矩阵映射乘法替换：

```
// 模糊度浮点解正变换: z = Z^T * a
{
    Eigen::Map<const Eigen::MatrixXd> matZ(Z, n, n);
    Eigen::Map<const Eigen::VectorXd> veca(a, n);
    Eigen::Map<Eigen::VectorXd> vecz(z, n);
    vecz = matZ.transpose() * veca;
}
```

(2) **动态内存分配轻量化**：在文件头部自实现了 `mat()`、`zeros()`、`eye()` 等小巧的辅助函数，完全脱离了对 `rtklib.h` 的头文件包含。

(3) **构建系统无缝纳管**：在 `.vscode/tasks.json` 中添加 `"lambda.cpp"`，与主工程一起参与联合编译，实现单命令零警告编译输出。

4.3　feat#5 模糊度 Ratio 检验与固定基线解算公式推导

4.3.1　一、Ratio 检验统计量

整数最小二乘仅能保证在数学距离上找到残差平方和最小的整数候选组，但并不能保证该整数解在统计意义上绝对可靠（可能由于观测粗差、周跳未检出或多路径效应造成伪最优）。因此，必须对解出的整数模糊度进行可靠性检验。

工程中最常用的检验方法为 Ratio 检验（R-Ratio 检验），其定义为次优候选整数解的二次加权残差与最优候选解的二次加权残差之比：

$$\text{Ratio} = \frac{R_2}{R_1} = \frac{(\hat{\mathbf{a}} - \check{\mathbf{a}}_2)^T \mathbf{Q}_{\hat{\mathbf{a}}\hat{\mathbf{a}}}^{-1} (\hat{\mathbf{a}} - \check{\mathbf{a}}_2)}{(\hat{\mathbf{a}} - \check{\mathbf{a}}_1)^T \mathbf{Q}_{\hat{\mathbf{a}}\hat{\mathbf{a}}}^{-1} (\hat{\mathbf{a}} - \check{\mathbf{a}}_1)} \tag{27}$$

在代码中，`lambda(n, 2, ...)` 函数输出的 `s[0]` 与 `s[1]` 即分别对应最优解残差 R_1 与次优解残差 R_2：

$$\text{ratio} = \frac{s[1]}{s[0]} \tag{28}$$

设定检验阈值（在 `config.ini` 中配置为 `RatioThres = 3.0`）：

- 若 Ratio ≥ 3.0：说明最优候选解显著优于次优候选解，具有极高置信度，判定**固定成功（Fix OK）**，接受最优整数解 $\check{\mathbf{a}} = \check{\mathbf{a}}_1$；
- 若 Ratio < 3.0：说明最优解与次优解区分度不足，固定失败，判定为**浮点解（Float）**，保持浮点解算结果。

4.3.2 二、固定基线向量更新与方差恢复推导

当整周模糊度成功固定为整数 $\check{\mathbf{a}}$ 后，该模糊度不再作为未知参数，而是作为已知确定量反代入平差方程，以约束并重新解算基线向量 $\mathbf{b}$。

根据分块参数最小二乘理论，平差法方程为：

$$\begin{bmatrix} \mathbf{N}_{bb} & \mathbf{N}_{ba} \\ \mathbf{N}_{ab} & \mathbf{N}_{aa} \end{bmatrix} \begin{bmatrix} \hat{\mathbf{b}} \\ \hat{\mathbf{a}} \end{bmatrix} = \begin{bmatrix} \mathbf{W}_b \\ \mathbf{W}_a \end{bmatrix} \tag{29}$$

其对应的参数协因数矩阵分块表示为：

$$\mathbf{Q}_{\hat{\mathbf{x}}\hat{\mathbf{x}}} = \begin{bmatrix} \mathbf{Q}_{\hat{b}\hat{b}} & \mathbf{Q}_{\hat{b}\hat{a}} \\ \mathbf{Q}_{\hat{a}\hat{b}} & \mathbf{Q}_{\hat{a}\hat{a}} \end{bmatrix} \tag{30}$$

其中 $\mathbf{Q}_{\hat{b}\hat{b}}$ 为基线坐标分量的协方差，$\mathbf{Q}_{\hat{a}\hat{a}}$ 为双差模糊度的协方差，$\mathbf{Q}_{\hat{b}\hat{a}}$ 为基线与模糊度之间的互协方差矩阵。

当模糊度固定为真值 $\check{\mathbf{a}}$ 后，根据条件平差原理（或最小方差条件均值公式），固定解基线向量 $\check{\mathbf{b}}$ 与其方差阵 $\mathbf{Q}_{\check{b}\check{b}}$ 的严密解析表达式为：

$$\check{\mathbf{b}} = \hat{\mathbf{b}} - \mathbf{Q}_{\hat{b}\hat{a}}\mathbf{Q}_{\hat{a}\hat{a}}^{-1}(\hat{\mathbf{a}} - \check{\mathbf{a}}) \tag{31}$$

$$\mathbf{Q}_{\check{b}\check{b}} = \mathbf{Q}_{\hat{b}\hat{b}} - \mathbf{Q}_{\hat{b}\hat{a}}\mathbf{Q}_{\hat{a}\hat{a}}^{-1}\mathbf{Q}_{\hat{a}\hat{b}} \tag{32}$$

式 (31) 的物理意义极其明确：通过模糊度的先验整数偏差 $(\hat{\mathbf{a}} - \check{\mathbf{a}})$，借助互协方差项 $\mathbf{Q}_{\hat{b}\hat{a}}\mathbf{Q}_{\hat{a}\hat{a}}^{-1}$ 对浮点基线 $\hat{\mathbf{b}}$ 进行高精度线性拉升；式 (32) 说明减去正定矩阵项后，固定解基线方差大幅下降（由分米级跃升至毫米级）。

4.4 arch 相对定位主流程中的调度与接口规划

为了将已改造完成的 `lambda.cpp` 无缝接入系统主循环，在 `RTK.cpp` 与 `FormSDEpochObs.cpp` 之间规划了如下解耦数据调度流：

(1) **在 `FormSDEpochObs.cpp` 中提供浮点矩阵导出**：在 `RTKFloat` 完成收敛后，除了保存坐标外，将法方程求逆得到的全局协方差矩阵 $\mathbf{Q}_{\hat{\mathbf{x}}\hat{\mathbf{x}}}$ 进行子块切片：

- 浮点基线：$\hat{\mathbf{b}} = \hat{\mathbf{x}}[0 \dots 2]$；
- 浮点模糊度向量：$\hat{\mathbf{a}} = \hat{\mathbf{x}}[3 \dots 3 + 2S - 1]$；

- 模糊度自协方差：$\mathbf{Q}_{\hat{a}\hat{a}} = \mathbf{Q}_{\hat{\mathbf{x}}\hat{\mathbf{x}}}[3\ldots3+2S-1, 3\ldots3+2S-1]$；
- 互协方差矩阵：$\mathbf{Q}_{\hat{b}\hat{a}} = \mathbf{Q}_{\hat{\mathbf{x}}\hat{\mathbf{x}}}[0\ldots2, 3\ldots3+2S-1]$。

(2) **在 RunRTK 中组织固定解主控闭环：**

```
// RunRTK 中的主调度逻辑架构
if (float_ok) {
    int n_amb = 2 * Raw.DDObs.Sats; // 双差模糊度参数总维数
    double F[2 * MAXAMB], s[2];

    // 1. 调用已适配的 LAMBDA 算法搜索整数候选解
    int info = lambda(n_amb, 2, a_hat.data(), Qaa.data(), F, s);

    if (info == 0 && s[0] > 0.0) {
        double ratio = s[1] / s[0];
        Raw.DDObs.Ratio = ratio;

        // 2. Ratio 检验与基线更新
        if (ratio >= cfg.RatioThres) {
            Raw.DDObs.bFixed = true;
            Eigen::VectorXd a_check = Eigen::Map<Eigen::VectorXd>(F, n_amb);

            // 3. 应用公式恢复固定基线: b_check = b_hat - Qba * Qaa^{-1} * (a_hat - a_check)
            Eigen::Vector3d b_check = b_hat - Qba * Qaa.inverse() * (a_hat - a_check);
            Raw.DDObs.dPos[0] = b_check(0);
            Raw.DDObs.dPos[1] = b_check(1);
            Raw.DDObs.dPos[2] = b_check(2);

            // 4. 更新流动站固定坐标: X_R = X_B + b_check
            RovRes.Position[0] = X_B[0] + b_check(0);
            RovRes.Position[1] = X_B[1] + b_check(1);
            RovRes.Position[2] = X_B[2] + b_check(2);
        }
    }
}
```

(3) **状态与日志输出：**在主循环日志输出行中追加 Ratio 指标与解状态标记（RTK Fix OK 或 RTK Float），在全流程结束时统计全历元固定率（Fix Rate），从而实现高精度相对定位的算法与工程闭环。